

Assignment 9.5

Name: kasuri Rohith

Hall no: 2303a510i2

Batch: 23

Problem 1: String Utilities Function

Consider the following Python function:

```
def reverse_string(text):  
  
    return text[::-1]
```

Task:

1. Write documentation in:

o (a) Docstring

o (b) Inline comments

o (c) Google-style documentation

2. Compare the three documentation styles.

3. Recommend the most suitable style for a utility-based string library.

Code:

(a) Using Docstring

```
1  ✓ def reverse_string(text):  
2  ✓     """  
3      Reverses the given string.  
4  
5      Args:  
6      |   text (str): Input string to reverse.  
7  
8      Returns:  
9      |   str: The reversed string.  
10     """  
11     return text[::-1]  
12  
13  
14  ✓ if __name__ == "__main__":  
15     print(reverse_string("Hello"))
```

(b) Inline comments

```
18 def reverse_string(text):
19     # Function to reverse a given string
20     # text: input string
21     # Returns: reversed string
22     return text[::-1] # Slicing method to reverse string
23
24
25 # Running the function
26 print(reverse_string("Hello"))
```

(c) Using Google-Style Documentation

```
48 def reverse_string(text):
49     """
50     Reverses the given string.
51     Args:
52     |     text (str): Input string to reverse.
53     Returns:
54     |     str: The reversed string.
55     """
56     return text[::-1]
57
58 # Running the function
59 print(reverse_string("Hello"))
```

Ouput:

```
Help on module demo1:

NAME
    demo1

FUNCTIONS
    reverse_string(text)
        Reverses the given string.

    Args:
        text (str): Input string to reverse.

    Returns:
        str: The reversed string.

FILE
    c:\users\rohit\links\demo1.py
-- More --
```

Problem 2: Password Strength Checker

Consider the function:

```
def check_strength(password):  
    return len(password) >= 8
```

Task:

1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

Code:

(a) Docstring Style

```

28 def check_strength(password):
29     """
30     Checks whether the given password is strong.
31     A password is considered strong if it has
32     at least 8 characters.
33
34     Parameters:
35     password (str): The password string.
36
37     Returns:
38     bool: True if strong, False otherwise.
39     """
40     return len(password) >= 8
41
42
43 # Running
44 print(check_strength("mypassword"))
45 print(check_strength("pass"))

```

(b) Inline Comments Style

```

61 ✓ def check_strength(password):
62 ✓     """
63     Checks whether the given password is strong.
64     A password is considered strong if it contains
65     at least 8 characters.
66
67     Args:
68     |     password (str): Password provided by the user.
69
70     Returns:
71     |     bool: True if password length is >= 8, else False.
72     """
73     return len(password) >= 8
74
75 # Running
76 print(check_strength("mypassword"))
77 print(check_strength("pass"))

```

```

79 def check_strength(password):
80     """
81     Checks whether the given password is strong.
82     A password is considered strong if it contains
83     at least 8 characters.
84
85     Args:
86     | password (str): Password provided by the user.
87
88     Returns:
89     | bool: True if password length is >= 8, else False.
90     """
91     return len(password) >= 8
92
93
94 # Running
95 print(check_strength("mypassword"))
96 print(check_strength("pass"))

```

Output:

```

❖ PS C:\Users\rohit\Links> python -m pydoc demo1
True
False
Help on module demo1:

NAME
    demo1

FUNCTIONS
FUNCTIONS
    check_strength(password)
        Checks whether the given password is strong.
        A password is considered strong if it has
        at least 8 characters.

        Parameters:
        password (str): The password string.

        Returns:
        bool: True if strong, False otherwise.

FILE
    c:\users\rohit\links\demo1.py
-- More --

```

Problem 3: Math Utilities Module

Task:

1. Create a module `math_utils.py` with functions:
 - o `square(n)`
 - o `cube(n)`
 - o `factorial(n)`
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.

Code:

```

1  def square(n):
2      """
3      Returns the square of a number.
4
5      Args:
6      |   n (int or float): Input number.
7
8      Returns:
9      |   int or float: Square of n.
10     """
11     return n * n
12
13
14  def cube(n):
15      """
16      Returns the cube of a number.
17
18      Args:
19      |   n (int or float): Input number.
20
21      Returns:
22      |   int or float: Cube of n.
23     """
24     return n * n * n
25
26
27  def factorial(n):
28      """
29      Returns the factorial of a non-negative integer.
30
31      Args:
32      |   n (int): Non-negative integer.
33
34      Returns:
35      |   int: Factorial of n.
36
37      Raises:
38      |   ValueError: If n is negative.
39     """
40     if n < 0:
41         raise ValueError("Factorial not defined for negative numbers")
42
43     result = 1
44     for i in range(1, n + 1):
45         result *= i
46     return result
47
48
49  if __name__ == "__main__":
50     print(square(4))
51     print(cube(3))
52     print(factorial(5))

```

Output:

```
PS C:\Users\rohit\Links> python -m pydoc demo1
Help on module demo1:

NAME
    demo1

FUNCTIONS
    cube(n)
        Returns the cube of a number.

        Args:
            n (int or float): Input number.

        Returns:
            int or float: Cube of n.

    factorial(n)
        Returns the factorial of a non-negative integer.

        Args:
            n (int): Non-negative integer.

        Returns:
            int: Factorial of n.

        Raises:
            ValueError: If n is negative.

    square(n)
        Returns the square of a number.

        Args:
            n (int or float): Input number.

        Returns:
            int or float: Square of n.

FILE
    c:\users\rohit\links\demo1.py

PS C:\Users\rohit\Links>
```

Problem 4: Attendance Management Module

Task:

1. Create a module attendance.py with functions:

o mark_present(student)

o mark_absent(student)

o get_attendance(student)

2. Add proper docstrings.

3. Generate and view documentation in terminal and browse

Code:


```

1  attendance_record = {}
2
3  def mark_present(student):
4      """
5          Marks a student as present.
6
7          Args:
8              student (str): Name of the student.
9
10             Returns:
11                 None
12             """
13      attendance_record[student] = "Present"
14
15
16  def mark_absent(student):
17      """
18          Marks a student as absent.
19
20          Args:
21              student (str): Name of the student.
22
23             Returns:
24                 None
25             """
26      attendance_record[student] = "Absent"
27
28
29  def get_attendance(student):
30      """
31          Retrieves the attendance status of a student.
32
33          Args:
34              student (str): Name of the student.
35
36             Returns:
37                 str: Attendance status ('Present', 'Absent', or 'Not Marked').
38             """
39      return attendance_record.get(student, "Not Marked")
40
41
42  if __name__ == "__main__":
43      mark_present("Raju")
44      mark_absent("Ramesh")
45
46      print(get_attendance("Raju"))
47      print(get_attendance("Ramesh"))
48      print(get_attendance("Suresh"))

```

Output:

```
PS C:\Users\rohit\Links> python -m pydoc demo1

NAME
    demo1

FUNCTIONS
    get_attendance(student)
        Retrieves the attendance status of a student.

        Args:
            student (str): Name of the student.

        Returns:
            str: Attendance status ('Present', 'Absent', or 'Not Marked').

    mark_absent(student)
        Marks a student as absent.

        Args:
            student (str): Name of the student.

        Returns:
            None

    mark_present(student)
        Marks a student as present.

        Args:
            student (str): Name of the student.

        Returns:
            None

    mark_present(student)
        Marks a student as present.

        Args:
            student (str): Name of the student.

        Returns:
            None
            student (str): Name of the student.

        Returns:
            None

DATA
    Returns:
        None

DATA

DATA
    attendance_record = {}

FILE
    c:\users\rohit\links\demo1.py

FILE
    c:\users\rohit\links\demo1.py

PS C:\Users\rohit\Links> 
```

Problem 5: File Handling Function

Consider the function:

```
def read_file(filename):  
    with open(filename, 'r') as f:  
        return f.read()
```

Task:

1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation.

```
1  def read_file(filename):  
2      """  
3          Reads and returns the content of a file.  
4  
5          Args:  
6              filename (str): Path to the file.  
7  
8          Returns:  
9              str: File contents as a string.  
10  
11         Raises:  
12             FileNotFoundError: If the file does not exist.  
13             OSError: If an OS-related error occurs.  
14         """  
15         with open(filename, 'r') as f:  
16             return f.read()  
17  
18  
19     # Running Example  
20     if __name__ == "__main__":  
21         print(read_file("sample.txt"))
```

```

1  ✓ def read_file(filename):
2  ✓     """
3      Reads and returns file content.
4
5      Args:
6      |     filename (str): File name.
7
8      Returns:
9      |     str: File data.
10
11     Raises:
12     |     FileNotFoundError: If file not found.
13     """
14  ✓     with open(filename, 'r') as f:
15      |         return f.read()
16
17
18  ✓ if __name__ == "__main__":
19  ✓     try:
20      |         print(read_file("sample.txt"))
21  ✓     except Exception as e:
22      |         print("Error:", e)

```

```

1  ✓ def read_file(filename):
2      # Function to read file content
3      # filename: name of the file
4      # May raise FileNotFoundError if file not found
5  ✓     with open(filename, 'r') as f: # open file in read mode
6      |         return f.read() # return entire content
7
8
9      # Running Example
10  ✓ if __name__ == "__main__":
11      |         print(read_file("sample.txt"))

```

Output:

- PS C:\Users\rohit\Links> python -m pydoc demo1
- Help on module demo1:

NAME
demo1

FUNCTIONS

read_file(filename)
Reads and returns the content of a file.

Args:
filename (str): Path to the file.

Returns:
str: File contents as a string.

Raises:
FileNotFoundError: If the file does not exist.
OSError: If an OS-related error occurs.

FILE
c:\users\rohit\links\demo1.py

❖ PS C:\Users\rohit\Links>